Tejaswi Khode BEB221

Experiment: Morphological Analysis and Word Generation

Aim

To perform **morphological analysis and word generation** for any given text.

Theory

Introduction

Morphology is the study of the structure and formation of words. In Natural Language Processing (NLP), **morphological analysis** is the process of breaking words into their basic meaningful components and identifying their grammatical properties.

For example, the words:

- `develop`
- `developed`
- `developing`
- `developer`
- `development`

are morphologically related because they are derived from the root word **develop**.

Morphological analysis is useful in NLP because the same concept can appear in a text in several different word forms. Identifying the base form of these words helps computers process and understand text more effectively.

Morphological Analysis

Morphological analysis identifies the **root word, base form, prefixes, suffixes, and grammatical information** associated with a word.

For example:

Word	Base Form	Morphological Information
developed	develop	Past tense verb
developing	develop	Present participle
developers	developer	Plural noun
projects	project	Plural noun
applications	application	Plural noun
managed	manage	Past tense verb

In this experiment, **lemmatization** is used to obtain the meaningful base form of words. Part-of-Speech (POS) tagging is also used to identify whether a word is a noun, verb, adjective, adverb, etc.

Word Generation

Word generation is the process of generating different forms of a word from a root or base word.

For example, from the root word **develop**, we can generate:

```
develop
develops
developed
developing
developer
development
```

Similarly, from **manage**, we can obtain:

```
manage
manages
managed
managing
manager
management
```

Word generation demonstrates how prefixes, suffixes and grammatical changes can produce different forms of a word.

NLP Techniques Used

1. Tokenization

Tokenization divides a sentence or document into smaller units called **tokens**.

For example:

```
Python developer has experience
```

becomes:

```
Python
developer
has
experience
```

2. Part-of-Speech Tagging

Part-of-Speech (POS) tagging assigns a grammatical category to each word.

Common POS tags include:

Tag	Meaning
NN	Noun
NNS	Plural Noun
VB	Verb
VBD	Past Tense Verb
VBG	Verb ending in -ing
JJ	Adjective
RB	Adverb
NNP	Proper Noun

For example:

```
developed → VBD
projects → NNS
Python → NNP
```

3. Lemmatization

Lemmatization converts a word into its meaningful **base or dictionary form**.

Examples:

text projects → project applications → application developed → develop managed → manage

Lemmatization is useful because it treats different grammatical forms of a word as related to the same base word.

4. Root Word

A root or base word represents the fundamental form from which other related words can be derived.

For example:

```
develop
   |
   ├── developed
   ├── developing
   ├── developer
   └── development
```

Applications

Morphological analysis and word generation can be used in many NLP applications:

1. **Resume Analysis** – identifying different forms of skills and job-related terms.
2. **Search Engines** – matching related forms of search terms.
3. **Information Retrieval** – improving document and keyword matching.
4. **Text Classification** – improving text representation.

Advantages

- Helps identify the base form of words.
- Reduces the number of different word forms that need to be processed.
- Improves text processing and information retrieval.
- Helps identify related words in resumes.

Limitations

- Lemmatization may require POS information for accurate results.
- Simple suffix-based word generation may produce invalid words.
- Some words have irregular morphological forms.

- Technical terms and abbreviations in resumes may not be handled perfectly.

In [1]:
```
!pip install nltk pandas
```

```
Requirement already satisfied: nltk in /home/computer/anaconda3/lib/python3.9/site-packages (3.7)
Requirement already satisfied: pandas in /home/computer/anaconda3/lib/python3.9/site-packages (1.4.2)
Requirement already satisfied: joblib in /home/computer/anaconda3/lib/python3.9/site-packages (from nltk)
(1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /home/computer/anaconda3/lib/python3.9/site-packages (f
rom nltk) (2022.3.15)
Requirement already satisfied: tqdm in /home/computer/anaconda3/lib/python3.9/site-packages (from nltk)
(4.64.0)
Requirement already satisfied: click in /home/computer/anaconda3/lib/python3.9/site-packages (from nltk)
(8.0.4)
Requirement already satisfied: python-dateutil>=2.8.1 in /home/computer/anaconda3/lib/python3.9/site-pack
ages (from pandas) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /home/computer/anaconda3/lib/python3.9/site-packages (from
pandas) (2021.3)
Requirement already satisfied: numpy>=1.18.5 in /home/computer/anaconda3/lib/python3.9/site-packages (fro
m pandas) (1.21.5)
Requirement already satisfied: six>=1.5 in /home/computer/anaconda3/lib/python3.9/site-packages (from pyt
hon-dateutil>=2.8.1->pandas) (1.16.0)
```

In [2]:
```
import pandas as pd
import nltk
import re

from nltk.tokenize import word_tokenize
from nltk import pos_tag
from nltk.stem import WordNetLemmatizer
```

In [3]:
```
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package punkt to /home/computer/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to
[nltk_data]     /home/computer/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /home/computer/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /home/computer/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package wordnet to /home/computer/nltk_data...
[nltk_data] Downloading package omw-1.4 to /home/computer/nltk_data...
```

Out[3]:
```
True
```

In [5]:
```
df = pd.read_csv("UpdatedResumeDataSet.csv")
```

In [6]:
```
print(df.head())
```

```
        Category                                             Resume
0  Data Science  Skills * Programming Languages: Python (pandas...
1  Data Science  Education Details \r\nMay 2013 to May 2017 B.E...
2  Data Science  Areas of Interest Deep Learning, Control Syste...
3  Data Science  Skills â¯¢ R â¯¢ Python â¯¢ SAP HANA â¯¢ Table...
4  Data Science  Education Details \r\n MCA   YMCAUST,  Faridab...
```

In [7]:
```
print(df.columns)
```

```
Index(['Category', 'Resume'], dtype='object')
```

In [9]:
```
print("Number of resumes:", len(df))
```

```
Number of resumes: 962
```

In [10]:
```
text = df['Resume'].iloc[0]

print(text)
```

```
Skills * Programming Languages: Python (pandas, numpy, scipy, scikit-learn, matplotlib), Sql, Java, JavaS
cript/JQuery. * Machine learning: Regression, SVM, NaÃ¯ve Bayes, KNN, Random Forest, Decision Trees, Boos
ting techniques, Cluster Analysis, Word Embedding, Sentiment Analysis, Natural Language processing, Dimen
sionality reduction, Topic Modelling (LDA, NMF), PCA & Neural Nets. * Database Visualizations: Mysql, Sql
Server, Cassandra, Hbase, ElasticSearch D3.js, DC.js, Plotly, kibana, matplotlib, ggplot, Tableau. * Othe
rs: Regular Expression, HTML, CSS, Angular 6, Logstash, Kafka, Python Flask, Git, Docker, computer vision
- Open CV and understanding of Deep learning.Education Details

Data Science Assurance Associate

Data Science Assurance Associate - Ernst & Young LLP
Skill Details
JAVASCRIPT- Exprience - 24 months
jQuery- Exprience - 24 months
Python- Exprience - 24 monthsCompany Details
company - Ernst & Young LLP
description - Fraud Investigations and Dispute Services   Assurance
TECHNOLOGY ASSISTED REVIEW
TAR (Technology Assisted Review) assists in accelerating the review process and run analytics and generat
e reports.
* Core member of a team helped in developing automated review platform tool from scratch for assisting E
discovery domain, this tool implements predictive coding and topic modelling by automating reviews, resul
ting in reduced labor costs and time spent during the lawyers review.
* Understand the end to end flow of the solution, doing research and development for classification model
s, predictive analysis and mining of the information present in text data. Worked on analyzing the output
s and precision monitoring for the entire tool.
* TAR assists in predictive coding, topic modelling from the evidence by following EY standards. Develope
d the classifier models in order to identify "red flags" and fraud-related issues.

Tools & Technologies: Python, scikit-learn, tfidf, word2vec, doc2vec, cosine similarity, NaÃ¯ve Bayes, LD
A, NMF for topic modelling, Vader and text blob for sentiment analysis. Matplot lib, Tableau dashboard fo
r reporting.

MULTIPLE DATA SCIENCE AND ANALYTIC PROJECTS (USA CLIENTS)
TEXT ANALYTICS - MOTOR VEHICLE CUSTOMER REVIEW DATA * Received customer feedback survey data for past one
year. Performed sentiment (Positive, Negative & Neutral) and time series analysis on customer comments ac
ross all 4 categories.
* Created heat map of terms by survey category based on frequency of words * Extracted Positive and Negat
ive words across all the Survey categories and plotted Word cloud.
* Created customized tableau dashboards for effective reporting and visualizations.
CHATBOT * Developed a user friendly chatbot for one of our Products which handle simple questions about h
ours of operation, reservation options and so on.
* This chat bot serves entire product related questions. Giving overview of tool via QA platform and also
give recommendation responses so that user question to build chain of relevant answer.
* This too has intelligence to build the pipeline of questions as per user requirement and asks the relev
ant /recommended questions.

Tools & Technologies: Python, Natural language processing, NLTK, spacy, topic modelling, Sentiment analys
is, Word Embedding, scikit-learn, JavaScript/JQuery, SqlServer

INFORMATION GOVERNANCE
Organizations to make informed decisions about all of the information they store. The integrated Informat
ion Governance portfolio synthesizes intelligence across unstructured data sources and facilitates action
to ensure organizations are best positioned to counter information risk.
* Scan data from multiple sources of formats and parse different file formats, extract Meta data informat
ion, push results for indexing elastic search and created customized, interactive dashboards using kiban
a.
* Preforming ROT Analysis on the data which give information of data which helps identify content that is
either Redundant, Outdated, or Trivial.
* Preforming full-text search analysis on elastic search with predefined methods which can tag as (PII) p
ersonally identifiable information (social security numbers, addresses, names, etc.) which frequently tar
geted during cyber-attacks.
Tools & Technologies: Python, Flask, Elastic Search, Kibana

FRAUD ANALYTIC PLATFORM
Fraud Analytics and investigative platform to review all red flag cases.
â¢ FAP is a Fraud Analytics and investigative platform with inbuilt case manager and suite of Analytics
for various ERP systems.
* It can be used by clients to interrogate their Accounting systems for identifying the anomalies which c
an be indicators of fraud by running advanced analytics
Tools & Technologies: HTML, JavaScript, SqlServer, JQuery, CSS, Bootstrap, Node.js, D3.js, DC.js
```

In [11]:
```
text = re.sub(r'[^A-Za-z\s]', ' ', text)
text = re.sub(r'\s+', ' ', text)

print(text[:1000])
```

```
Skills Programming Languages Python pandas numpy scipy scikit learn matplotlib Sql Java JavaScript JQuery
Machine learning Regression SVM Na ve Bayes KNN Random Forest Decision Trees Boosting techniques Cluster
Analysis Word Embedding Sentiment Analysis Natural Language processing Dimensionality reduction Topic Mod
elling LDA NMF PCA Neural Nets Database Visualizations Mysql SqlServer Cassandra Hbase ElasticSearch D js
DC js Plotly kibana matplotlib ggplot Tableau Others Regular Expression HTML CSS Angular Logstash Kafka P
ython Flask Git Docker computer vision Open CV and understanding of Deep learning Education Details Data
Science Assurance Associate Data Science Assurance Associate Ernst Young LLP Skill Details JAVASCRIPT Exp
rience months jQuery Exprience months Python Exprience monthsCompany Details company Ernst Young LLP desc
ription Fraud Investigations and Dispute Services Assurance TECHNOLOGY ASSISTED REVIEW TAR Technology Ass
isted Review assists in accelerating the review proce
```

In [12]:
```
tokens = word_tokenize(text)

print("First 50 words:")
print(tokens[:50])
```

```
First 50 words:
['Skills', 'Programming', 'Languages', 'Python', 'pandas', 'numpy', 'scipy', 'scikit', 'learn', 'matplotl
ib', 'Sql', 'Java', 'JavaScript', 'JQuery', 'Machine', 'learning', 'Regression', 'SVM', 'Na', 've', 'Baye
s', 'KNN', 'Random', 'Forest', 'Decision', 'Trees', 'Boosting', 'techniques', 'Cluster', 'Analysis', 'Wor
d', 'Embedding', 'Sentiment', 'Analysis', 'Natural', 'Language', 'processing', 'Dimensionality', 'reducti
on', 'Topic', 'Modelling', 'LDA', 'NMF', 'PCA', 'Neural', 'Nets', 'Database', 'Visualizations', 'Mysql',
'SqlServer']
```

In [13]:
```
pos_words = pos_tag(tokens)

print(pos_words[:50])
```

```
[('Skills', 'NNS'), ('Programming', 'VBG'), ('Languages', 'NNP'), ('Python', 'NNP'), ('pandas', 'VBZ'),
('numpy', 'JJ'), ('scipy', 'JJ'), ('scikit', 'NN'), ('learn', 'NN'), ('matplotlib', 'NN'), ('Sql', 'NN
P'), ('Java', 'NNP'), ('JavaScript', 'NNP'), ('JQuery', 'NNP'), ('Machine', 'NNP'), ('learning', 'VBG'),
('Regression', 'NNP'), ('SVM', 'NNP'), ('Na', 'NNP'), ('ve', 'FW'), ('Bayes', 'NNP'), ('KNN', 'NNP'), ('R
andom', 'NNP'), ('Forest', 'NNP'), ('Decision', 'NNP'), ('Trees', 'NNP'), ('Boosting', 'NNP'), ('techniqu
es', 'NNS'), ('Cluster', 'NNP'), ('Analysis', 'NNP'), ('Word', 'NNP'), ('Embedding', 'NNP'), ('Sentimen
t', 'NNP'), ('Analysis', 'NNP'), ('Natural', 'NNP'), ('Language', 'NNP'), ('processing', 'NN'), ('Dimensi
onality', 'NNP'), ('reduction', 'NN'), ('Topic', 'NNP'), ('Modelling', 'NNP'), ('LDA', 'NNP'), ('NMF', 'N
NP'), ('PCA', 'NNP'), ('Neural', 'NNP'), ('Nets', 'NNP'), ('Database', 'NNP'), ('Visualizations', 'NNP'),
('Mysql', 'NNP'), ('SqlServer', 'NNP')]
```

In [14]:
```
lemmatizer = WordNetLemmatizer()
```

In [15]:
```
lemmatized_words = []

for word, tag in pos_words:
    lemma = lemmatizer.lemmatize(word.lower())
    lemmatized_words.append((word, lemma))
```

In [25]:
```
print("Original Word -> Lemma")

for original, lemma in lemmatized_words[:20]:
    print(original, "->", lemma)
```

```
Original Word -> Lemma
Skills -> skill
Programming -> programming
Languages -> language
Python -> python
pandas -> panda
numpy -> numpy
scipy -> scipy
scikit -> scikit
learn -> learn
matplotlib -> matplotlib
Sql -> sql
Java -> java
JavaScript -> javascript
JQuery -> jquery
Machine -> machine
learning -> learning
Regression -> regression
SVM -> svm
Na -> na
ve -> ve
```

In [17]:
```
from nltk.corpus import wordnet

def get_wordnet_pos(tag):
    if tag.startswith('J'):
        return wordnet.ADJ
    elif tag.startswith('V'):
        return wordnet.VERB
    elif tag.startswith('N'):
        return wordnet.NOUN
    elif tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN
```

In [18]:
```
morphological_results = []

for word, tag in pos_words:
    wn_tag = get_wordnet_pos(tag)
    lemma = lemmatizer.lemmatize(word.lower(), wn_tag)
    morphological_results.append((word, tag, lemma))
```

In [26]:
```
print("Word\t\tPOS\tLemma")
print("-" * 40)

for word, tag, lemma in morphological_results[:20]:
    print(f"{word:15} {tag:5} {lemma}")
```

```
Word            POS     Lemma
----------------------------------------
Skills          NNS   skill
Programming     VBG   program
Languages       NNP   language
Python          NNP   python
pandas          VBZ   pandas
numpy           JJ    numpy
scipy           JJ    scipy
scikit          NN    scikit
learn           NN    learn
matplotlib      NN    matplotlib
Sql             NNP   sql
Java            NNP   java
JavaScript      NNP   javascript
JQuery          NNP   jquery
Machine         NNP   machine
learning        VBG   learn
Regression      NNP   regression
SVM             NNP   svm
Na              NNP   na
ve              FW    ve
```

In [20]:
```
def generate_words(root):
    words = [
        root,
        root + "s",
        root + "ed",
        root + "ing",
        root + "er",
        root + "ment"
    ]
    return words
```

In [21]:
```
root = "develop"

generated = generate_words(root)

print("Root word:", root)
print("Generated words:")

for word in generated:
    print(word)
```

```
Root word: develop
Generated words:
develop
develops
developed
developing
developer
development
```

In [27]:
```
roots = ["develop", "manage", "program", "analyze", "create"]

for root in roots:
    print("\nRoot:", root)
    print(generate_words(root))
```

```
Root: develop
['develop', 'develops', 'developed', 'developing', 'developer', 'development']

Root: manage
['manage', 'manages', 'manageed', 'manageing', 'manageer', 'management']

Root: program
['program', 'programs', 'programed', 'programing', 'programer', 'programment']

Root: analyze
['analyze', 'analyzes', 'analyzeed', 'analyzeing', 'analyzeer', 'analyzement']

Root: create
['create', 'creates', 'createed', 'createing', 'createer', 'createment']
```

In [28]:
```
word_families = {
    "develop": [
        "develop",
        "develops",
        "developed",
        "developing",
        "developer",
        "development"
    ],

    "manage": [
        "manage",
        "manages",
        "managed",
        "managing",
        "manager",
        "management"
    ],

    "program": [
        "program",
        "programs",
        "programmed",
        "programming",
        "programmer"
    ],

    "analyze": [
        "analyze",
        "analyzes",
        "analyzed",
        "analyzing",
        "analysis",
        "analyst"
    ]
}
```

In [29]:
```
for root, words in word_families.items():
    print("\nRoot Word:", root)
    print("Generated Forms:", ", ".join(words))
```

```
Root Word: develop
Generated Forms: develop, develops, developed, developing, developer, development

Root Word: manage
Generated Forms: manage, manages, managed, managing, manager, management

Root Word: program
Generated Forms: program, programs, programmed, programming, programmer

Root Word: analyze
Generated Forms: analyze, analyzes, analyzed, analyzing, analysis, analyst
```

In [ ]: